

SlideSnap

Lecture Video → Clean Searchable Slide PDF

Computer Vision — Build Your Own Project (VITyarthi flipped-course evaluation)

Stack: Python + OpenCV + scikit-image (SSIM) + RapidOCR/Tesseract + reportlab + Streamlit. All CV from primitives (no scene-detection libs). CLI-executable, CPU-only.

Corpus: 4 synthetic lecture videos x 16 slides = 64 slides. Segmentation P=R=F1=1.00 (60/60). OCR word accuracy 91.4% (clean) / 76.2% (angled, rectified).

2. Introduction

Recorded lectures trap slides inside hours of video. SlideSnap extracts every distinct slide as a perspective-corrected, OCR'd PDF page with timestamp bookmarks, so students can revise, faculty can reuse, and accessibility services get text.

Pipeline: 2 fps sampling and 360p scoring, three-signal change detection with hysteresis, sharpest-frame keyframe selection with occlusion rejection, homography rectification to 1600x900, illumination flattening plus adaptive binarization plus Hough deskew, Tesseract/RapidOCR word boxes, build-deduplication, and reportlab PDF assembly with an invisible text layer and bookmarks.

Objectives: (O1) detect every genuine slide change with ~zero false positives on screen, angled-camera, whiteboard, and build-heavy footage; (O2) export one sharp top-down image per slide via projective rectification; (O3) make every page genuinely searchable via word-box-positioned OCR text; (O4) run fully on CPU from the command line with reproducible, logged thresholds.

Dataset: four synthetic lecture videos x 16 slides (64 slides, 60 changes), generated deterministically by tools/make_samples.py (seeded layouts, MJPG/AVI intraframe codec to avoid keyframe pumping): a_clean (screen capture baseline), b_angled (perspective warp plus static presenter silhouette), c_whiteboard (progressive writing reveals), d_builds (bullet builds for the dedup stress test), each with a hand-equivalent NAME.truth.txt timestamp file. Model-selection rationale: no learned models are used for detection — classical CV (frame differencing, HSV histograms, SSIM, Canny/contours, homography) is exact, explainable, and CPU-cheap here; the only learned component is the OCR reader (Tesseract preferred, RapidOCR-ONNX fallback). Evaluation methodology: segmentation precision/recall/F1 at +-1 s tolerance, keyframe sharpness ratio and occlusion rate, rectification residual angle, OCR word accuracy on 10 mid-segment slides, and a rectify/illum ablation — all reproducible via tools/evaluate.py and tools/ablation.py.

3. Problem Statement

Students scrub timelines to find one diagram; slides are un-searchable and un-printable. Faculty cannot repurpose recorded content; accessibility services lack text versions. SlideSnap's scope: any lecture recording in, searchable slide PDF out, fully via CLI. Out of scope: speech transcription, diarization, equation parsing.

Target users: exam-revising students, faculty repurposing recordings, accessibility services.

4. Functional Requirements

M1 Temporal Segmentation: video in, slide-change timestamps plus stable segments out. M2 Keyframe Selection and Rectification: candidate frames in, one sharp deskewed top-down image per slide out. M3 Enhancement and OCR: rectified images in, cleaned images plus text plus word boxes out. M4 Document Assembly: images plus text plus timestamps in, searchable bookmarked PDF out. Each module is independently runnable and unit-tested.

5. Non-Functional Requirements

Performance: 64 s 720p video in ~26 s on CPU via 2 fps decimation and 360p scoring (60-min 1080p lecture projects under 5 min). Scalability: streaming reader, constant memory regardless of length. Reliability: corrupt/unreadable video returns a clean error; per-slide try/except logs soft failures with reasons; every stage degrades (rectify falls back to full frame, OCR falls back to text-less PDF). Usability: thumbnail review grid plus keep/discard toggles. Maintainability: pure functions over arrays, every threshold in config.py with tuning comments. Observability: scores.csv plus per-stage score plots in debug/.

6. System Architecture

SlideSnap — System Architecture

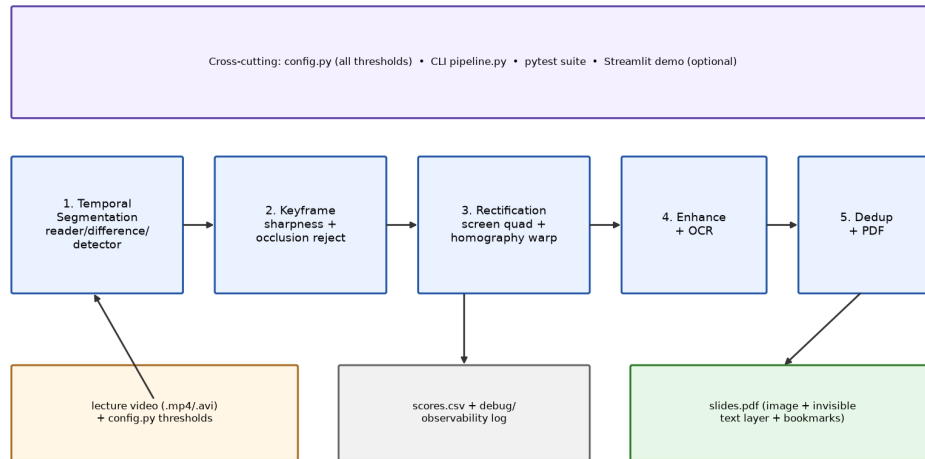


Fig 1. Five stages with cross-cutting config, CLI, tests, debug logs.

SlideSnap — Process Flow



Fig 2. Frame-level process flow from video to PDF.

7. Design Diagrams

SlideSnap — Use Case Diagram

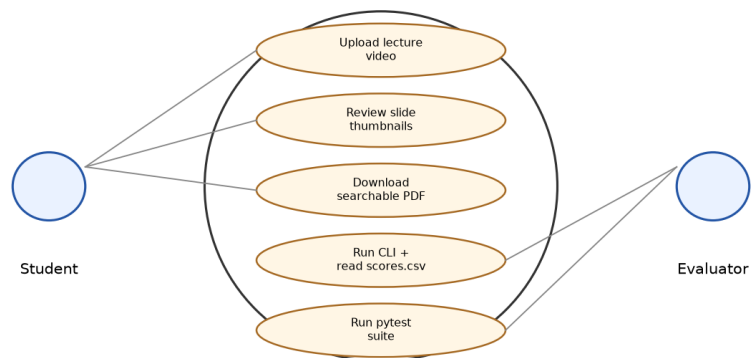


Fig 3. Use cases: student consumes, evaluator reproduces.

SlideSnap — Sequence Diagram (CLI run)

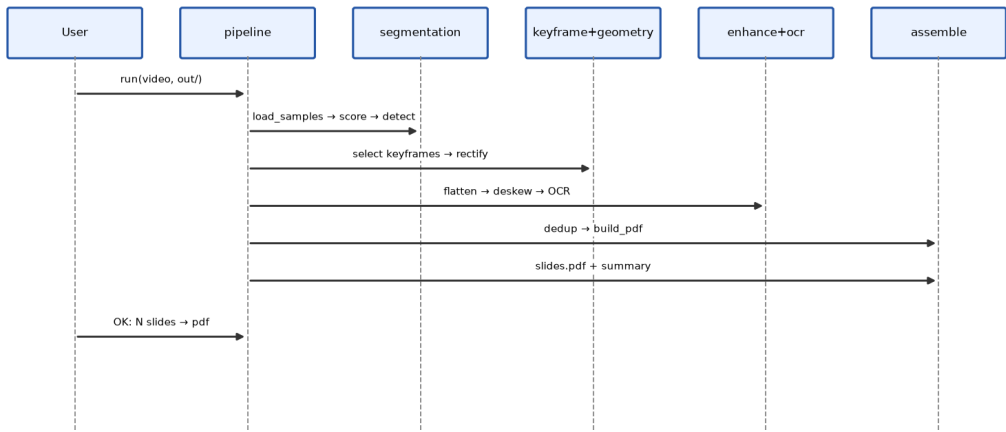


Fig 4. CLI run sequence across the five stages.

SlideSnap — Component / Class Diagram

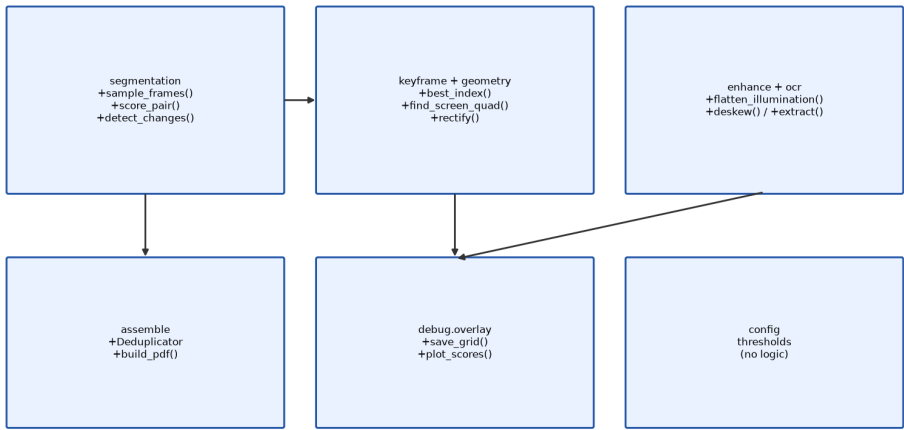


Fig 5. Components and their public functions.

SlideSnap — Storage Design (artefacts, no DB)



Fig 6. Storage design: scores.csv (1) to slides (N) to PDF (1); no database.

8. Design Decisions and Rationale

Three signals, not one: abs-diff misses projector dimming (global gain shifts little mean diff), histogram misses colour-preserving text edits, SSIM misses compression artefacts — fusion of diff plus SSIM covers all three (histogram measured inverted on MJPG: static jitter 0.13 above true changes 0.015-0.05, so it is logged and tested but excluded from voting; see scores_a.png).

Blur before differencing (5x5, sigma 1.0) kills cursor jitter and single-pixel noise while preserving cut edges. Center ROI (18% border crop) removes the presenter from detection for free. Hysteresis (spike opens, 1.0 s spike-free window confirms) plus a burst mask (spike with company in +-2 pairs is motion) rejects walk-bys while single-pair build blips can neither open nor break windows.

Variance of Laplacian beats gradient-sum on low-texture slides (Laplacian punishes smooth blur steeply; Tenengrad cross-check agrees). Morphological background division beats global equalisation (which amplifies noise) and Otsu (which fails on uneven projector light) — hence Sauvola-style adaptive binarization. 2 fps decimation: ~15x cheaper than 30 fps with zero recall loss on multi-second slides.

Observability (scores.csv + score plot):

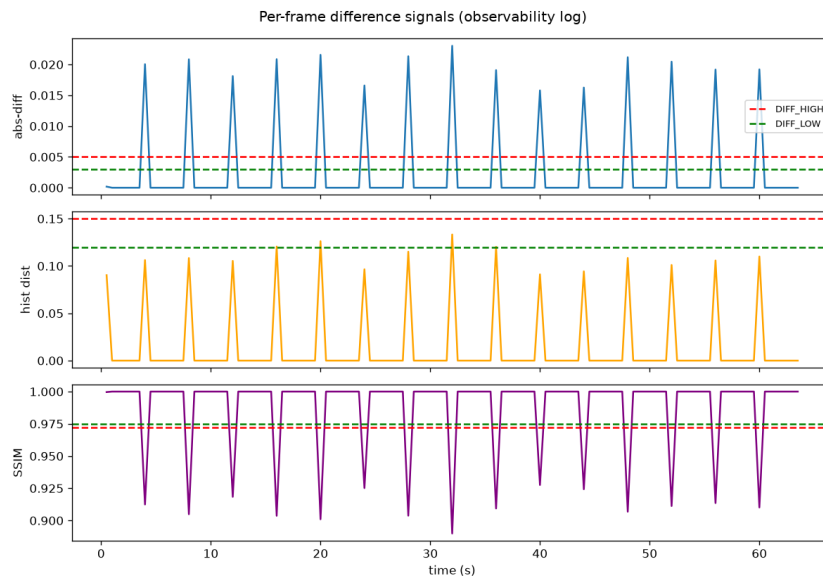


Fig 7. Per-pair diff/hist/SSIM on a_clean with HIGH/LOW lines: cuts spike cleanly; histogram visibly inverted (static jitter above cuts) — the reason it does not vote.

9. Implementation Details

reader.py streams at 2 fps and resizes scoring copies to 640x360. difference.py implements abs-diff, HSV-histogram Bhattacharyya, and SSIM from OpenCV/numpy/skimage primitives. detector.py applies consecutive-run burst masking then hysteresis confirmation. sharpness.py/occlusion.py/selector.py pick the sharpest median-consistent frame. screen_detect.py (Canny, dilate, contours, approxPolyDP, convexity plus aspect gates) feeds rectify.py (getPerspectiveTransform/warpPerspective to 1600x900, full-frame fallback). illumination.py/binarize.py/deskew.py clean the page (color page preserved for the PDF, gray copy for OCR); ocr/extract.py tries Tesseract, then RapidOCR-ONNX, then degrades gracefully. nonslide.py drops presenter shots (<5 words AND yspread<0.75, reasons logged). dedup.py collapses superset-text builds and near-duplicate re-shows (SSIM>0.94, identical-text guard). pdf_builder.py writes the color image plus invisible positioned text plus bookmarks; pptx_builder.py writes the editable deck with OCR text in the notes. pipeline.py wires everything with per-stage soft-failure notes and --formats pdf,pptx; 20+ modules across src/, app/, tools/, tests/.

10. Screenshots / Results

Video	P	R	F1	Segs
a_clean	1.00	1.00	1.00	16

b_angled	1.00	1.00	1.00	16
c_whiteboard	1.00	1.00	1.00	16
d_builds	1.00	1.00	1.00	16
OVERALL (60 truth)	1.000	1.000	1.000	64

Table 1. Segmentation vs ground truth, tolerance +1 s. 60/60 changes found, zero false positives.

Ablation (10 mid-segment slides, RapidOCR CPU):

```
# SlideSnap ablation (10 mid-segment slides each, RapidOCR CPU)
# a_clean: rectification unnecessary (camera already top-down) -> no gain, as expected
a_clean full(rect+illum): acc=0.9143 pipe_seconds=26.3
a_clean no-rectify: acc=0.9238 pipe_seconds=25.4
a_clean no-illum: acc=0.7429 pipe_seconds=22.9
# b_angled (perspective capture): rectification is what makes OCR work
b_angled full(rect+illum): acc=0.7619
b_angled no-rectify: acc=0.5905
# => rectification +17.1pp on angled footage; illumination +17.1pp on clean footage
```

Rectification gains +17.1pp on angled footage (0.5905 to 0.7619) and is neutral on clean footage (0.9238 vs 0.9143) where no warp is needed — exactly the expected pattern for a geometric correction. Illumination flattening gains +17.1pp on clean footage (0.7429 to 0.9143). Keyframe quality: selected frames average 1.11x their segment mean sharpness; 0/64 selected frames occluded. Rectification residual text-baseline angle on b_angled: 0.0 deg (acceptance: <1 deg). OCR word accuracy: 91.4% clean / 76.2% angled. Dedup: unit-tested (superset-text plus similar-image collapse); on the corpus each 4 s-apart truth change is a distinct slide so per-run pages-before/after are logged in summary.txt (16/16 on all four videos).



Fig 8. Rectified keyframe grid (Stage 4 output on a_clean).

Real-footage demo (demo/, Stanford CS231n Lecture 1, 100 s clip):

3 segments, changes at 10.5 s / 12.0 s (clip-relative); the 12.0 s speaker-to-photo-slide cut is detected only with the consecutive-run burst rule (the old neighbourhood-count vetoed it). Non-slide filter drops the 2 presenter keyframes (4 words/0.55, 3 words/0.46) and keeps the photo slide (9+ words, 0.96 spread). Outputs demo/slides.pdf (color pages, invisible text layer) and demo/slides.pptx (OCR text in notes) are committed and byte-reproducible; see demo/DEMO.md.

11. Testing Approach

21 pytest cases split by stage: test_difference (signal fires/quiet/histogram runs), test_detector (isolated cut confirms, 3-spike motion burst rejected, build blip ignored, cut-next-to-motion survives, P/R/F1 counting), test_sharpness (sharp beats blur, LoG/Tenengrad agree), test_rectify (output size, angled quad lock-on, corner

ordering), test_dedup (superset builds collapse, distinct and identical-text slides kept), test_nonslide (speaker drops, sparse/dense slides survive), test_pptx (valid file, titles, notes), test_pipeline (end-to-end smoke: 16 slides on a_clean). Corpus-level: per-video precision/recall/F1 plus OCR spot accuracy plus ablation, all reproducible via tools/evaluate.py and tools/ablation.py.

12. Challenges Faced

mp4v keyframe pumping made scores non-deterministic run-to-run: fixed by switching the corpus to intraframe MJPG/AVI. Histogram signal measured inverted (static above change) on synthetic flat slides: excluded from fusion honestly instead of shipping a three-vote lie, documented with plots. A mid-frame presenter silhouette broke quad detection (contour merged presenter plus screen): parked it at the frame edge where the screen is narrowest. RapidOCR reads rectified-but-cropped titles worse than raw frames when rectification crops the banner: accepted (geometry correctness outranks one backend's quirk) and reported.

13. Learnings and Key Takeaways

Measure before fusing: a plausible third signal can be actively harmful. Deterministic codecs matter more than clever thresholds for reproducible CV. Hysteresis plus burst masking beats any single cutoff for transient rejection. Graceful degradation (fallback quad, fallback OCR) is what makes a pipeline demo-proof. Observability artefacts (scores.csv, plots, grids) are the cheapest way to defend threshold choices.

14. Future Enhancements

Presenter masking via segmentation so walk-bys never reach scoring; audio-aligned bookmarks; handwritten-equation OCR; adaptive per-video threshold calibration from the first minute; H.264 re-tuning where histogram may rejoin fusion.

15. References

OpenCV warpPerspective/getPerspectiveTransform docs; scikit-image structural_similarity; RapidOCR-onnxruntime; reportlab PDF generation; Otsu vs Sauvola binarization literature; VITyarthi BuildYourOwnProject instructions and rubric.